

Scenario 4 — Premium Plan, Claim Rejected by Oracle

Full mechanics: see Step-1 through Step-4. Step 5 never happens in this scenario — that's the point.

An honest number problem, addressed directly

With the *actual* current numbers in this project (Premium plan coverage = 10_000 ether, oracle mock threshold = 100_000), a real oracle rejection via the amount threshold **cannot occur** for any claim that would pass the coverage check first — 10,000 never reaches 100,000. Rather than fabricate a claim amount that contradicts the real deployed code, this scenario makes an explicit, labeled assumption: **for this scenario only**, assume oracle-service's `MOCK_APPROVAL_THRESHOLD` constant was configured to 200 instead of its default 100,000 — a legitimate deployment-time configuration choice (it's a plain constant in `verificationLogic.js`), used here specifically to demonstrate what a rejection actually looks like end-to-end.

Values used

	Value
Policyholder	Alice, 0x1111...aaaa
templateId	3 (Premium), same policy as Scenario 3
Claim amount	221 ether (\$221) — same as Scenario 3
Assumed mock threshold (this scenario only)	\$200
Oracle decision	Rejected — $221 > 200$

Walkthrough (deltas from Step 4 onward)

- **Steps 1-3:** identical to Scenario 3 — claim 1 is created with status `Submitted`.
- **Step 4, `evaluateClaim`:** $221 > 200$ (the assumed threshold for this scenario) leads to `{approved: false, reason: "MOCK: declaredAmount 221 exceeds placeholder threshold 200"}`.
- **Step 4, `signOracleResponse`:** signs `{ claimId: 1, requestId: 0x2b3c4d..., approved: false, timestamp: ... }` — the EIP-712 mechanics are byte-for-byte identical to the approved case; only the `approved` field's value differs.
- **Step 4, `OracleAdapter.submitVerification`:** succeeds exactly as before — signature verification doesn't care what the decision was, only that it's genuinely signed by `0x8888...bbbb`. Calls `claimRegistry.recordOracleVerification(1, 0x2b3c4d..., false)`.
- **Step 4, `ClaimRegistry.recordOracleVerification`:** sets `claims[1].status = Rejected` (not `Approved`).

End state — and why Step 5 is unreachable

```
claims[1].status: Rejected
```

If the underwriter Safe attempted `ClaimRegistry.payoutClaim(1)` now, it would revert with `ClaimNotApproved` — `payoutClaim`'s very first check is `status == Approved`, and `Rejected` fails that check. No dUSD moves; Alice keeps her \$33 premium's worth of coverage active but

receives nothing for this specific claim. She could still file a different claim against the same policy later, as long as cumulative non-rejected claims stay within the \$10,000 coverage — and this rejected claim, once `Rejected`, no longer counts against that coverage at all (see `_consumedCoverage` 's logic in `Step-3`).